

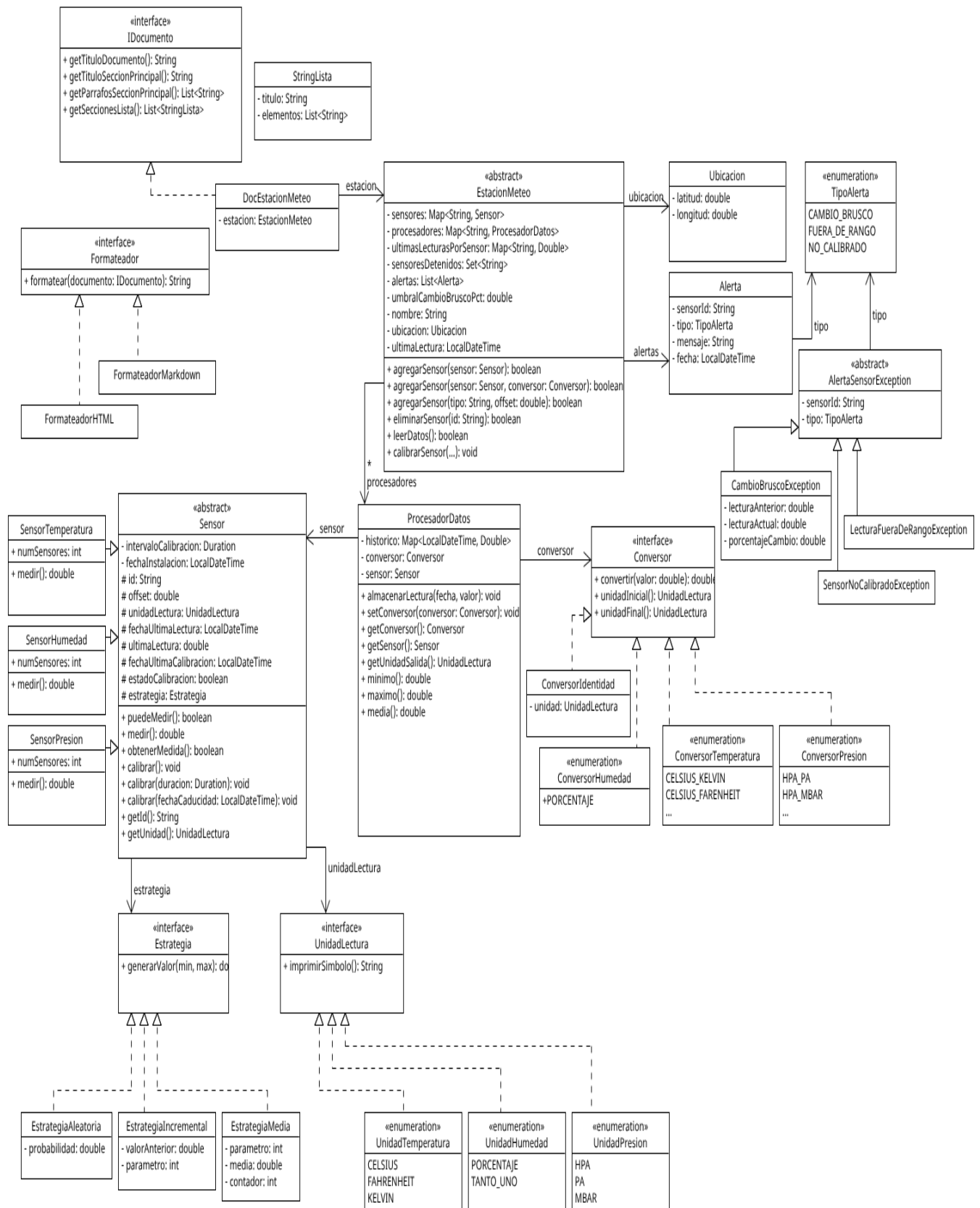
Análisis y Diseño de Software:

Práctica 4

Fernando Blanco Varela

Alejandro González García

Diagrama de clases:



Decisiones de Diseño:

1. Uso de herencia para los sensores:

Se ha definido una clase abstracta Sensor de la que heredan distintos tipos de sensores.

Justificación:

Permite **reutilizar comportamiento común** (id, lecturas, etc.).

Facilita añadir nuevos tipos de sensores sin modificar el código existente.

2. Uso del patrón Strategy en las lecturas

Las estrategias de lectura se encapsulan en clases independientes (por ejemplo, lectura instantánea, media, etc.).

Justificación:

Permite **cambiar el comportamiento en tiempo de ejecución**

Evita condicionales complejos.

Hace el sistema **más flexible y extensible**.

3. Separación entre procesamiento y almacenamiento

La clase ProcesadorDatos se encarga del tratamiento de datos, mientras que los sensores solo gestionan lecturas.

Justificación:

Mejora la **claridad del diseño**.

Permite **reutilizar el procesamiento** con distintos sensores.

4. Uso de interfaces para formateadores y documentos

Se han definido interfaces como Formateador e IDocumento.

Justificación:

Permite tener **múltiples implementaciones** (HTML, Markdown, etc.)

Facilita la separación entre generación de datos y su representación

Mejora la **extensibilidad sin modificar código existente**

5. Gestión de alertas separada

Las alertas se gestionan mediante clases independientes.

Justificación:

Permite añadir nuevas reglas de alerta sin afectar al resto del sistema
Mejora la **mantenibilidad**

6. Uso de enumeraciones para estados o niveles

Se utilizan enum para representar estados (por ejemplo, niveles de alerta o unidades).

Justificación:

Mejora la **legibilidad y seguridad de tipos**
Evita errores por uso de valores inválidos

7. Estructura de paquetes

El proyecto está organizado en paquetes (sensor, procesador, formateador, etc.).

Justificación:

Mejora la organización del código
Facilita la navegación y mantenimiento